

lab7

要求：实现这些测试样例：

`test_dup`、`test_dup2`、`test_pipe`、`test_open`、`test_openat`、`test_close`、
`test_getdents`、`test_read`、`test_mkdir`、`test_chdir`、`test_unlink`、`test_mount`、
`test_umount`、`test_fstat`

之前在lab3已经顺便实现了很多调用，因此这里只剩这些测试样例：

`test_dup`、`test_dup2`、`test_pipe`、`test_getdents`、`test_mkdir`、`test_chdir`、
`test_unlink`、`test_mount`、`test_umount`

▼ 前置操作：

- 添加这几个测试点

```
char *tests[] = {  
  
    // lab7  
    "dup",  
    "dup2",  
    "pipe",  
    "getdents",  
    "mkdir_", // 注意这里有一个_!!!!  
    "chdir",  
    "unlink",  
    "mount",  
    "umount",  
  
};
```

- 执行make fast_renew以后，把未实现的调用号先进行注册，相应的系统调用则先用return 0占位以确保可以通过编译。

```
#define SYS_chdir 49 // 把之前已有的进行修改  
...  
#define SYS_dup3 24  
#define SYS_mkdirat 34  
#define SYS_getdents 61
```

```
#define SYS_unlink 35
#define SYS_mount 40
#define SYS_umount 39
```

▼ dup

- 默认实现好了 `dup`，只需要对应注册，以及在 `init.c` 中添加测试点即可通过。

▼ dup2

- 看一下要做什么

```
void test_dup2(){
    TEST_START(__func__);
    int fd = dup2(STDOUT, 100);
    assert(fd != -1);
    const char *str = "    from fd 100\n";
    write(100, str, strlen(str));
    TEST_END(__func__);
}
```

在帮助文档中https://github.com/oscomp/testsuite-for-oskernel/blob/main/oscomp_syscalls.md，我们知道，这里的 `dup2` 测试样例会传递 `dup3` 调用号，然后进行 `sys_dup3` 的系统调用，而这个调用相比之前的 `dup`，则实现了，将第一个参数的文件复制到指定文件描述符的功能。

- 看看原版 `dup` 是怎么做的

```
uint64
sys_dup(void)
{
    struct file *f;
    int fd;

    if (argfd(0, 0, &f) < 0)
        return -1;
    if ((fd = fdalloc(f)) < 0)
        return -1;
    filedup(f);
}
```

```

    return fd;
}

```

首先将传入的第一个参数当做文件描述符来解析，用 `argfd` 直接将解析完的指向真实文件的指针传到第三个参数里，随后用 `fdalloc`，在当前进程中的 `p->ofile[]` 找一个空闲的位置安放这个文件指针，并返回 `ofile` 的索引作为新的文件描述符，最后为 `f` 这个文件增加一次引用。

- 因此在原版 `dup` 的基础上，我们只需要将 `fdalloc` 干的事情变成直接指向存放即可，因此我们不能再直接调用 `fdalloc` 了。

```

// Allocate a file descriptor for the given file.
// Takes over file reference from caller on success.
static int
fdalloc(struct file *f)
{
    int fd;
    struct proc *p = myproc();

    for (fd = 0; fd < NOFILE; fd++)
    {
        if (p->ofile[fd] == 0)
        {
            p->ofile[fd] = f;
            return fd;
        }
    }
    return -1;
}

```

但是你注意到，原版的 `fdalloc` 中，所有文件描述符会有一个上限 `NOFILE`，因此在该进程的所有文件描述符，都不可能超过这个 `NOFILE`。在测试样例中，传入了100这个文件描述符作为指定，但 `NOFILE` 却只有16，如果我们直接做，就会失败。所以在 `param.h` 中，我们需要修改这个宏定义：`#define NOFILE 110`，之后，再直接写好 `sys_dup3` 即可

```

uint64
sys_dup3(void)
{

```

```

struct file *f;
int oldfd, newfd;

if (argfd(0, 0, &f) < 0)
    return -1;
if (argint(1, &newfd) < 0)
    return -1;
if (argint(0, &oldfd) < 0)
    return -1;
struct proc *p = myproc();
if (newfd < 0 || newfd >= NOFILE || p->ofile[newfd]
    != NULL)
    return -1;
p->ofile[newfd] = f;
filedup(f);
return newfd;
}

```

▼ pipe

- 添加测试点后即可直接通过。

▼ getdents

- 看看要做什么吧

```

char buf[512];
void test_getdents(void){
    TEST_START(__func__);
    int fd, nread;
    struct linux_dirent64 *dirp64;
    dirp64 = buf;
    //fd = open(".", O_DIRECTORY);
    fd = open(".", O_RDONLY);
    printf("open fd:%d\n", fd);

    nread = getdents(fd, dirp64, 512);
    printf("getdents fd:%d\n", nread);
    assert(nread != -1);
    printf("getdents success.\n%s\n", dirp64->d_nam

```

```

e);

    /*
    for(int bpos = 0; bpos < nread;){
        d = (struct dirent *)(buf + bpos);
        printf( "%s\t", d->d_name);
        bpos += d->d_reclen;
    }
    */

    printf("\n");
    close(fd);
    TEST_END(__func__);
}

```

根据文档的描述，这里调用 `getdents` 传入的第二个参数 `dirp64` 是这样的一个结构，以 `linux_dirent64` 结构的指针解析 `buf` 这段空数组

```

struct dirent {
    uint64 d_ino;    // 索引结点号
    int64 d_off;     // 到下一个dirent的偏移
    unsigned short d_reclen;    // 当前dirent的长度
    unsigned char d_type;    // 文件类型
    char d_name[];    // 文件名
};

```

成功执行之后，则会将读出的字节数返回给 `nread`，最后，用 `dirp64` 指针解析 `buf` 区，把 `d_name` 输出。

- 这里我们就需要了解一下内核中的 `dirent` 是什么，这个在lab3中有提到过一点，简短来说：`struct dirent` 实际上代表着在物理磁盘上，这个文件的真实情况，也就是说操作系统的全局中，它的信息只对应一个文件，这是唯一的，又称为这个文件的元数据，我们只是用 `struct file` 作为打开文件表，指向该物理数据。

在 `fat32.h` 中，我们阅读这个结构体的真实模样，发现它和要求传入的结构不同。

```

struct dirent {
    char filename[FAT32_MAX_FILENAME + 1];
};

```

```

uint8    attribute;
// uint8    create_time_tenth;
// uint16   create_time;
// uint16   create_date;
// uint16   last_access_date;
uint32   first_clus;
// uint16   last_write_time;
// uint16   last_write_date;
uint32   file_size;

uint32   cur_clus;
uint      clus_cnt;

/* for OS */
uint8    dev;
uint8    dirty;
short    valid;
int      ref;
uint32   off;           // offset in the parent d
ir entry, for writing convenience
    struct dirent *parent; // because FAT32 doesn't
have such thing like inum, use this for cache trick
    struct dirent *next;
    struct dirent *prev;
    struct sleeplock lock;
};

```

它的大小高达360Bytes，而我们需要的却是经过解析后的若干个
[linux_dirent64](#) 小结构体。这里添加一个新的定义。

```

struct linux_dirent64
{
    uint64 d_ino;
    uint64 d_off;
    unsigned short d_reclen;
    unsigned char d_type;
    char d_name[0];
};

```

- 事实上 `getdents` 的功能，就是遍历并读取一个打开的目录中的内容，并将目录项（文件、子目录等）的信息填充到用户态提供的缓冲区中。而这里的内容，则转化为上面这个小结构体作为存储办法，一点一点填充到 `buf` 中。

由于用户态的linux标准，需要使用VFS这一套标准逻辑，但在内核态中，却是FAT32的文件系统，它不存在 `d_ino`（inode）这种概念，因此要进行标准化处理，我们就需要引入 `getdents` 作为“欺上瞒下”的处理过程。

```
uint64
sys_getdents(void)
{
    struct file *f;
    uint64 buf;
    int len;

    if (argfd(0, 0, &f) < 0 || argaddr(1, &buf) < 0
    || argint(2, &len) < 0)
        return -1;

    if (f->readable == 0 || !(f->ep->attribute & ATT
R_DIRECTORY))
        return -1;

    struct dirent de;
    int count = 0;
    int ret;
    int nread = 0;
    struct linux_dirent64 lde;

    ...

    return nread;
}
```

开头我们先进行一些基本操作，把参数解析了，并且声明一些必要变量。这里的 `lde` 作为我们一步一步解析后的标准结构体，每次承担转化与转存到 `buf` 的功能，`nread` 则表示读取了多少byte。

- 接下来我们要进行解析的过程。这里我们要先看一个关键函数，它能帮我们在FAT32这个文件系统中解析 `dirent` 。

```
int enext(struct dirent *dp, struct dirent *ep, uint off, int *count)
{
    if (!(dp->attribute & ATTR_DIRECTORY))
        panic("enext not dir");
    if (ep->valid)
        panic("enext ep valid");
    if (off % 32)
        panic("enext not align");
    if (dp->valid != 1) { return -1; }

    union dentry de;
    int cnt = 0;
    memset(ep->filename, 0, FAT32_MAX_FILENAME + 1);
    for (int off2; (off2 = reloc_clus(dp, off, 0)) != -1; off += 32) {
        if (rw_clus(dp->cur_clus, 0, 0, (uint64)&de, off2, 32) != 32 || de.lne.order == END_OF_ENTRY) {
            return -1;
        }
        if (de.lne.order == EMPTY_ENTRY) {
            cnt++;
            continue;
        } else if (cnt) {
            *count = cnt;
            return 0;
        }
        if (de.lne.attr == ATTR_LONG_NAME) {
            int lcnt = de.lne.order & ~LAST_LONG_ENTRY;
            if (de.lne.order & LAST_LONG_ENTRY) {
                *count = lcnt + 1;
            }
        }
        // plus the s-n-e;
    }
}
```



```

        count = 0;
    }
    read_entry_name(ep->filename + (lcnt -
1) * CHAR_LONG_NAME, &de);
    } else {
        if (count) {
            *count = 1;
            read_entry_name(ep->filename, &d
e);
        }
        read_entry_info(ep, &de);
        return 1;
    }
}
return -1;
}

```

FAT32文件系统中，所有的文件条目顺序排序，正常情况下会存成这样的结构，这里只给文件名留了11个字节的大小，总体的大小则正好是32字节。

```

typedef struct short_name_entry {
    char        name[CHAR_SHORT_NAME];
    uint8       attr;
    uint8       _nt_res;
    uint8       _crt_time_tenth;
    uint16      _crt_time;
    uint16      _crt_date;
    uint16      _lst_acce_date;
    uint16      fst_clus_hi;
    uint16      _lst_wrt_time;
    uint16      _lst_wrt_date;
    uint16      fst_clus_lo;
    uint32      file_size;
} __attribute__((packed, aligned(4))) short_name_e
ntry_t;

```

如果长过11字节的名字，则会由另一个结构体 `long_name_entry_t` 经过连续的存放来保管名字碎片。

而 `enext` 则替我们把这个机制隔绝了，这样，在 `enext` 运行时，经过了多少个 `entry`，就会把 `count` 设为几。但FAT32中，用户删除某一文件，只会将 `entry` 的第一个字节设为 `EMPTY_ENTRY`，但 `enext` 不会在遇到空洞时直接跳过，而是正常将 `count++`，这是为了后续运行下一次 `enext` 之前，内核可以知道真正有用的下一个 `entry` 在什么位置。总的来说，`enext` 做了三件事：跳空洞、拼长名、提核心属性，最后将相关内容存放到传来的参数 `de` 中。

所以，现在我们就可以围绕这个 `enext` 得到我们的 `getdents` 了

```
uint64
sys_getdents(void)
{
    struct file *f;
    uint64 buf;
    int len;

    if (argfd(0, 0, &f) < 0 || argaddr(1, &buf) < 0
        || argint(2, &len) < 0)
        return -1;

    if (f->readable == 0 || !(f->ep->attribute & ATT
R_DIRECTORY))
        return -1;

    struct dirent de;
    int count = 0;
    int ret;
    int nread = 0;
    struct linux_dirent64 lde;

    elock(f->ep);
    while (1)
    {
        ret = enext(f->ep, &de, f->off, &count);
        if (ret == 0)
        { // 这个文件已被删除，直接跳过
```

```

        f->off += count * 32;
        continue;
    }
    if (ret == -1) // 后面不再有目录了
        break;

    int name_len = strlen(de.filename);
    int reclen = 19 + name_len + 1; // 19 是 struct linux_dirent64不包含文件名大小的size
    reclen = (reclen + 7) & ~7; // align to 8

    if (nread + reclen > len)
    {
        break; // 再存进去一个linux_dirent64会超过buf的最大空间
    }

    f->off += count * 32;

    // 这下面的部分只是为了严谨起见，但完全可以不添加这些内容
    lde.d_ino = de.first_clus; // 用首簇号作为inode
    lde.d_off = f->off; // 如果还想getdents，下次从哪里开始读
    lde.d_reclen = reclen; // 包含真实文件名长度的该结构体大小
    lde.d_type = (de.attribute & ATTR_DIRECTORY) ? 4 : 8; // 目录文件则为4，普通文件为8

    // 这下面就是存储过程了
    // 先把不包含文件名的内容存一遍到buf
    if (copyout2(buf + nread, (char *)&lde, 19) < 0)
    {
        eunlock(f->ep);
        return -1;
    }
    // 再把文件名补充进去

```

```

        if (copyout2(buf + nread + 19, de.filename, name_len + 1) < 0)
        {
            eunlock(f->ep);
            return -1;
        }

        nread += reclen; // 更新现在读了多少字节
    }
    eunlock(f->ep);

    return nread;
}

```

- 这样，最后在输出中，我们会看到 `bin`，也就是从 `.` 开始向下解析出来的第一个 `dirent` 名字，而这里得到的504，则是linux对于8的对齐，这时，就无需再像FAT32那样32对齐了。

▼ mkdir

- 看一下要做什么

```

void test_mkdir(void){
    TEST_START(__func__);
    int rt, fd;

    rt = mkdir("test_mkdir", 0666);
    printf("mkdir ret: %d\n", rt);
    assert(rt != -1);
    fd = open("test_mkdir", O_RDONLY | O_DIRECTORY);
    if(fd > 0){
        printf("  mkdir success.\n");
        close(fd);
    }
    else printf("  mkdir error.\n");
    TEST_END(__func__);
}

```

这里会调用 `mkdirat` 而非 `mkdir`，它有更强的能力，可以实现：如果 `path` 是相对路径，则它是相对于 `dirfd` 目录而言的。如果 `path` 是相对路径，且 `dirfd` 的值为 `AT_FDCWD`，则它是相对于当前路径而言的。如果 `path` 是绝对路径，则 `dirfd` 被忽略。

如果我们看一下具体的调用情况，实际上只需要考虑传的 `dirfd` 为 `AT_FDCWD` 的情况即可。

```
int mkdir(const char *path, mode_t mode)
{
    return syscall(SYS_mkdirat, AT_FDCWD, path, mode);
}
```

- 所以我们可以基于原有 `mkdir` 的基础上修改，得到 `mkdirat`。

先看看 `mkdir`

```
uint64
sys_mkdir(void)
{
    char path[FAT32_MAX_PATH];
    struct dirent *ep;

    if (argstr(0, path, FAT32_MAX_PATH) < 0 || (ep = create(path, T_DIR, 0)) == 0)
    {
        return -1;
    }
    eunlock(ep);
    eput(ep);
    return 0;
}
```

直接利用 `path` 创建一个目录，这里的 `create` 逻辑就是很简单地找到 `path` 对应的 `parent` 节点的 `dirent`，然后在这个 `dirent` 下从物理磁盘上 `ealloc` 一个新的 `entry`，但是完全没有考虑相对路径的事。所以我们首先修改参数解析逻辑：

```
uint64
sys_mkdirat(void)
```

```

{
    char path[FAT32_MAX_PATH];
    struct dirent *ep;
    int flags;

    if (argstr(1, path, FAT32_MAX_PATH) < 0 || argint(
2, &flags) < 0 || (ep = create(path, T_DIR, flags))
== NULL)
    {
        return -1;
    }
    eunlock(ep); // 解开睡眠锁，允许其他进程访问它
    eput(ep);
    return 0;
}

```

之后，通过相对路径解析绝对路径的事情，我们在 `openat` 测试点其实已经实现过一次了，所以接下来的逻辑也都差不多，直接实现就好了。

```

uint64
sys_mkdirat(void)
{
    char path[FAT32_MAX_PATH];
    struct dirent *ep;

    // 接下来将path转换为绝对路径，方法和sys_open里处理相对路
    径的方法一样
    int dirfd, mode;

    if (argint(0, &dirfd) < 0 || argstr(1, path, FAT32_
MAX_PATH) < 0 || argint(2, &mode) < 0)
    {
        return -1;
    }

    if (path[0] == '/' || dirfd == AT_FDCWD)
    { // 绝对路径或者没有指定 dirfd (AT_FDCWD)，直接使用 pat
    h 进行查找

```

```

    ;
}
else
{
    struct file *dirf;
    if (argfd(0, 0, &dirf) < 0 || !(dirf->ep->attribute & ATTR_DIRECTORY))
    {
        return -1;
    }

    struct dirent *curr = dirf->ep;
    char fullpath[FAT32_MAX_PATH];
    char *p_out = fullpath + FAT32_MAX_PATH - 1;
    *p_out = '\0';

    int path_len = strlen(path);
    if (path_len >= FAT32_MAX_PATH)
        return -1;
    p_out -= path_len;
    memmove(p_out, path, path_len);

    while (curr != NULL && curr->parent != curr && curr->parent != NULL)
    {
        int len = strlen(curr->filename);
        if (p_out - fullpath < len + 1)
        {
            return -1;
        }
        p_out--;
        *p_out = '/';
        p_out -= len;
        memmove(p_out, curr->filename, len);
        curr = curr->parent;
    }

    if (p_out > fullpath)

```

```

    {
        p_out--;
        *p_out = '/';
    }
    safestrcpy(path, p_out, FAT32_MAX_PATH);
}

if ((ep = create(path, T_DIR, mode)) == NULL)
{
    return -1;
}

eunlock(ep); // 解开睡眠锁，允许其他进程访问它
eput(ep);
return 0;
}

```

▼ chdir

- 实现完 `mkdir` 以后添加这个测试样例即可通过。

▼ unlink

- 先看一下要做什么

```

int test_unlink()
{
    TEST_START(__func__);

    char *fname = "./test_unlink";
    int fd, ret;

    fd = open(fname, O_CREATE | O_WRONLY);
    assert(fd > 0);
    close(fd);

    // unlink test
    ret = unlink(fname);
    assert(ret == 0);
}

```



```

    fd = open(fname, O_RDONLY);
    if(fd < 0){
        printf("  unlink success!\n");
    }else{
        printf("  unlink error!\n");
        close(fd);
    }
    // It's Ok if you don't delete the inode and data
    blocks.

    TEST_END(__func__);
}

```

可以看到 `unlink` 的作用实际上与删除文件的作用类似，但需要进行链接的删除，如果全部链接都消失，这个文件就被删除了。之后，我们发现它最终调用的是 `unlinkat`，只不过在测试样例中，依然使用 `AT_FDCWD` 忽略了其他情况。所以我们还是需要解析相对路径。

- 在传统的 xv6（基于 Inode 和 EXT2 思想）中，`unlink` 只是把文件的“硬链接数（link count）”减 1，只有当链接数降为 0 且没有进程打开它时，才会真正删除数据。**但是，目前所处的 FAT32 文件系统的世界里，根本没有“硬链接”这个概念！**在 FAT32 中，`unlink` 就是极其直接的“物理抹杀”。它的核心动作只有两个：
 - **立墓碑**：把父目录中记录这个文件的条目（Directory Entry）的第一个字节改成 `0xE5`。
 - **扬骨灰**：顺着 FAT 表，把这个文件占用的所有数据簇（Clusters）全部标记为空闲。
- 所以全部的流程大概是：提取参数，但这里我们只需要处理相对当前路径这一种情况；获取到路径对应的 `dirent`，检查该 `dirent` 是否为一个非空的目录；不是则直接删除，之前我们也说过，事实上 FAT32 删除文件的办法就是把 `entry` 的头一个字节设置为 `EMPTY_ENTRY`。而在 `fat.c` 中，有一些封装好的函数可以帮助。
- 事实上，在这里的 `unlink`，其逻辑可以完全套用 `sys_remove` 的逻辑。

```

uint64
sys_unlink(void)
{

```

```

char path[FAT32_MAX_PATH];
struct dirent *ep;
int len;

if ((len = argstr(1, path, FAT32_MAX_PATH)) <= 0)
    return -1;

char *s = path + len - 1;
while (s >= path && *s == '/')
{
    s--;
}
// 这里拒绝删除 "." 和 "..", 否则会导致文件系统混乱 (虽然
// FAT32 本身就没有这两个目录项, 但为了兼容性和安全性, 我们也不允
// 许用户创建它们)
if (s >= path && *s == '.' && (s == path || *--s ==
'/'))
{
    return -1;
}

// 获取文件对应的 dirent
// 注意: 和前面mkdirat的情况一样, 这里简单的 ename(path)
// 在 dirfd == AT_FDCWD 时
// 刚好能够通过 myproc()->cwd 或者绝对路径来正确解析并帮我们
// 们通过这个测试点。
if ((ep = ename(path)) == NULL)
{
    return -1;
}

elock(ep);
// 如果是目录, 而且目录非空, 不能删除 (unlink主要针对文件,
// 但有些文件系统统一处理)
if ((ep->attribute & ATTR_DIRECTORY) && !isdirempty
(ep))
{
    eunlock(ep);
}

```

```

    eput(ep);
    return -1;
}

// 按照 FAT32 原理，将项目标记为0xE5（实际上eremove实现了这一步）
elock(ep->parent);
eremove(ep);
eunlock(ep->parent);

eunlock(ep);
eput(ep);

return 0; // unlink 成功返回0
}

```

▼ mount/umount

- 检查要做的事情之后，我们发现其实只需 `return 0` 就可以完全通过了。